BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2
NGÀNH: CÔNG NGHỆ KỸ THUẬT ĐIỆN, ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN HỌC PHẦN

MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI:

TÌM VỊ TRÍ ĐẦU, CUỐI VÀ TẦN SUẤT CỦA X

Mã đề: DT028

Giảng viên hướng dẫn	:	Hồ Nhựt Minh
Sinh viên thực hiện	:	PHAN ĐỨC HUY
MSSV	:	N24DCDT034
Lớp	:	D24CQDT01-N

TP. HCM, tháng 10 năm 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Thông tin đề tài và sinh viên
Họ tên MSSV lớp	Phan Đức Huy — N24DCDT034 — D24CQDT01-N
Mã đề và tên đề	DT028 — Tìm vị trí đầu, cuối và tần suất của x
Căn cứ giáo trình	Chương 2, Mục 2.6.2; mục 2.8 bài 28; trang in 50–52, 62; trang PDF 62–64, 74
Ngày giao ngày nộp	Ngày giao 17/10/2026 — Ngày nộp 20/10/2026

Đề bài: Cho mảng đã tăng có khóa trùng, tìm lần xuất hiện đầu tiên và cuối cùng bằng hai tìm kiếm nhị phân. Từ hai biên đếm số lần xuất hiện, phân biệt với tìm một vị trí bất kỳ và xử lý giá trị hoàn toàn vắng mặt. Trọng tâm là tự viết hàm, theo dõi biến qua từng bước và giải thích kết quả trên các trường hợp nhỏ.

Dữ liệu vào: a::Vector{Int} tăng không giảm, $0 \le n \le 10000$; x nguyên; vắng trả first = last = 0, count = 0. Nhập từ bàn phím hoặc truyền trực tiếp các giá trị, mảng đã khai báo trong chương trình.

Kết quả ra: Hai chỉ số 1-based đầu/cuối và tần suất; nếu thấy, count = last − first + 1; trace hai quá trình tìm biên.

Ví dụ 1: Đoạn chỉ số 2..4 gồm đúng ba số 2.

Ví dụ 2: Không gán vị trí chèn của 3 thành một lần xuất hiện.

Hướng dẫn thực hiện: 1. Đọc mục sách được chỉ dẫn; xác định điều kiện đầu vào, kết quả cần trả và giải ví dụ thứ nhất bằng tay.
2. Lần vết khi gặp a[mid] == x: bản tìm đầu tiếp tục về trái, bản tìm cuối tiếp tục về phải; vẫn lưu vị trí hiện có.
3. Cài hai hàm biên và hàm ghép kết quả; chỉ tính last − first + 1 khi đã xác nhận tìm thấy x.
4. Đối chiếu quét tuyến tính trên mảng nhỏ; thử x ở hai đầu, chiếm cả mảng và không có.
5. Chạy hai ví dụ và năm ca kiểm thử; đối chiếu kết quả mong đợi, sửa lỗi và giải thích một trường hợp biên.

Các hàm nên viết: vi_tri_dau(a,x); vi_tri_cuoi(a,x); tan_suat(a,x).

Năm ca cần kiểm tra: 1. [1, 2, 2, 2, 4], x = 2 cho 2, 4, 3.
2. Cùng mảng, x = 3 cho 0, 0, 0.
3. [5, 5, 5], x = 5 cho 1, 3, 3.

4. [1, 2], x = 1 cho 1, 1, 1.
5. Mảng rỗng cho 0, 0, 0.

Phân tích và nhận xét: Mỗi hàm O(log n), hai hàm vẫn O(log n); vùng phụ O(1). Bản quét tham chiếu O(n). Thử n = 5, 50, 500, 5000 với dãy nhiều trùng; đếm lần xét mid của từng biên riêng.

Yêu cầu thêm sau khi phần cơ bản chạy đúng: Tùy chọn: dùng hai biên để đếm số phần tử nằm trong đoạn giá trị [L, U] hợp lệ.

Sản phẩm: Mã nguồn, các kết quả chạy ví dụ và ca kiểm tra, một lần vết bằng tay, báo cáo theo mẫu môn học. Chạy từ bàn phím hoặc truyền trực tiếp các giá trị vào hàm.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

LỜI CAM ĐOAN

Em xin chịu trách nhiệm về toàn bộ nội dung của báo cáo này. Phần đọc giáo trình, hai hàm tìm biên, bộ kiểm thử, các lần chạy và ảnh chụp đều do em trực tiếp thực hiện; số liệu trong các bảng được lấy nguyên từ log của những lần chạy đó, không chỉnh sửa lại. Những chỗ có dựa vào tài liệu hay mã mẫu của giảng viên đều được ghi rõ nguồn ở phần tài liệu tham khảo, và em không sử dụng bài làm của sinh viên khác.

TP. Hồ Chí Minh, ngày 23 tháng 10 năm 2026

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

PHAN ĐỨC HUY

LỜI CẢM ƠN

Bài làm này hoàn thành được nhờ hướng dẫn của thầy Hồ Nhựt Minh: thầy chỉ cách đọc mục 2.6.2, cách chọn ca biên cho trường hợp khóa trùng và cách trình bày bảng truy vết sao cho người đọc dò lại được từng bước. Em cũng cảm ơn quý thầy cô đã giảng phần tìm kiếm nhị phân trong học phần, nhờ đó em hiểu vì sao phải tách hẳn hai lần tìm thay vì gọi một hàm có sẵn.

TÓM TẮT

Báo cáo giải bài toán tìm vị trí đầu, vị trí cuối và tần suất của khóa x trong một mảng số nguyên đã sắp tăng. Cách làm là cài tay hai lần tìm nhị phân: lần thứ nhất thu hẹp về bên trái mỗi khi gặp x để lấy biên đầu, lần thứ hai thu hẹp về bên phải để lấy biên cuối; tần suất chỉ được tính bằng last − first + 1 sau khi đã xác nhận x có mặt. Số liệu chạy thật cho thấy mỗi lần tìm không quá $\lceil \log_2(n+1) \rceil$ lần xét mid, cộng lại vẫn O(log n) và không tốn thêm bộ nhớ phụ. Bài nộp gồm mã Julia, bộ kiểm thử 13 ca có mã (12 ca chức năng và 1 ca dữ liệu sai), 11 ảnh chụp từ lần chạy thật và báo cáo theo khung của môn.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG iii
CÁCH ĐỐI CHIẾU RUBRIC iv
LỜI CAM ĐOAN v
LỜI CẢM ƠN vi
TÓM TẮT vii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 1
1.4. Cách thực hiện và cấu trúc báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 4
2.1. Khái niệm và mô hình dữ liệu 4
2.2. Cấu trúc dữ liệu sử dụng 4
2.3. Nguyên lý thuật toán 5
2.4. Cơ sở đánh giá thời gian và bộ nhớ 5
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 6
3.1. Phân tích yêu cầu chương trình 6
3.2. Thiết kế các hàm và dữ liệu 6
3.3. Thuật toán và mã nguồn chính 7
3.4. Lần vết ví dụ nhỏ 8
3.5. Giải thích tính đúng và điều kiện dừng 9
3.6. Phân tích độ phức tạp 10
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 11
4.1. Môi trường và kế hoạch kiểm thử 11
4.2. Bảng kết quả 13 ca 11
4.3. Hai ví dụ trên phiếu 14
4.4. Hai ca sinh viên tự thiết kế 15
4.5. Ba ca biên phù hợp với đề tài 16
4.6. Kiểm tra input không hợp lệ — LOI01 17
4.7. Kết quả toàn bộ kiểm thử 18
4.8. Khảo sát bốn quy mô hợp lệ 18
4.9. GitHub và chạy lại từ bản clone sạch 20
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 21
5.1. Mức hoàn thành so với phiếu 21

5.2. Giới hạn và bài học.....21
5.3. Hướng phát triển phù hợp.....21
TÀI LIỆU THAM KHẢO.....22
PHỤ LỤC.....PL-1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy.....PL-1
PHỤ LỤC B. Danh mục bằng chứng.....PL-2
PHỤ LỤC C. Tự kiểm trước khi nộp.....PL-3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM.....PL-4
PHỤ LỤC E — HAI ẢNH A10 VÀ A11 PL-6

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
a	Mảng số nguyên đã tăng không giảm (Vector{Int})
n	Số phần tử của mảng, $0 \le n \le 10000$
x	Khóa cần tìm vị trí đầu, cuối và tần suất
first	Chỉ số 1-based của lần xuất hiện đầu tiên; 0 nếu vắng
last	Chỉ số 1-based của lần xuất hiện cuối cùng; 0 nếu vắng
count	Tần suất: last − first + 1 khi x có mặt, ngược lại 0
lo, hi	Hai biên của vùng còn xét trong tìm kiếm nhị phân
mid	Vị trí giữa vùng: lo + (hi − lo) ÷ 2
kq	Vị trí biên tạm, 0 hoặc một vị trí có a[kq] = x
O(log n)	Thời gian của một lần tìm kiếm nhị phân
O(1)	Bộ nhớ phụ của thuật toán
PASS / FAIL	Kết luận của một ca kiểm thử

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào.....2
Bảng 1.2 Đặc tả kết quả đầu ra.....2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm.....3
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì.....4
Bảng 3.1 Tổ chức các hàm và tệp.....7
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm.....7
Bảng 3.3 Truy vết ca.....8
Bảng 3.4 Chi phí của từng pha trong bản cài.....10
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu.....11
Bảng 4.2 Thiết kế 13 ca kiểm độc lập.....12
Bảng 4.3 Actual kết luận và vị trí minh chứng.....13
Bảng 4.4 Khảo sát Q1–Q4.....19
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại.....20
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu.....21
Bảng PL.1 Bốn tệp cần nộp.....PL-4

DANH SÁCH HÌNH

Hình 4.1 — VD01: kết quả in ra từ demo.jl --ca=1, chụp trong VS Code.....14
Hình 4.2 — VD02: x vắng mặt nên kết quả là 0, 0, 0.....14
Hình 4.3 — TK01: x nằm ở đầu mảng.....15
Hình 4.4 — TK02: x nằm ở cuối mảng.....15
Hình 4.5 — BIEN01: mảng chỉ có hai phần tử.....16
Hình 4.6 — BIEN02: mảng rỗng.....16
Hình 4.7 — BIEN03: mọi phần tử đều bằng x.....17
Hình 4.8 — LOI01: ba kiểu dữ liệu sai đều bị từ chối.....17
Hình 4.9 — Tổng kết: 18.624 trên 18.624 phép kiểm đạt.....18
Hình 4.10 — A10: bản clone tại commit 4277eb8 chạy Run_all.ps1, kết thúc bằng END_TO_END_OK..... PL-6
Hình 4.11 — A11: kho Mildsivan/ctdl-baocao mở đúng tại commit 4277eb8.....PL-7

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Với dãy số nguyên đã sắp tăng a[1..n] và một khóa x, bài toán đặt ra ba câu hỏi nối tiếp nhau: x xuất hiện lần đầu ở đâu, lần cuối ở đâu, và giữa hai mốc đó có bao nhiêu phần tử. Cả ba câu trả lời đều dựa trên hai lần tìm kiếm nhị phân tự viết, nên chỗ khó không nằm ở công thức mà ở việc giữ đúng các biến lo, hi, mid, kq qua từng vòng và không lẫn vị trí chèn với một lần xuất hiện thật. Phần lý thuyết được lấy từ giáo trình INT1306, chương 2, mục 2.6.2 và bài 28 mục 2.8 (trang in 50–52 và 62).

1.2. Mục tiêu và sản phẩm

Sáu việc phải làm, kèm sản phẩm tương ứng:

• vi_tri_dau(a, x): trả chỉ số 1-based của lần xuất hiện đầu tiên, trả 0 khi x vắng mặt.

• vi_tri_cuoi(a, x): trả chỉ số của lần xuất hiện cuối cùng, cũng trả 0 khi x vắng mặt.

• tan_suat(a, x): ghép hai biên vừa tìm, trả first, last, count kèm số lần xét mid của mỗi lần.

• kiem_tra_dau_vao: chặn mảng sai kiểu, mảng không tăng và mảng dài quá 10000 phần tử.

• quet_tuyen_tinh làm bản đối chứng O(n); dem_trong_doan cài phần tùy chọn đếm phần tử trong đoạn giá trị [L, U].

• Bộ kiểm thử vượt mức 12 ca, ảnh chụp từ lần chạy thật, báo cáo và kho mã sao cho người khác chạy lại được.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Đầu vào gồm hai phần: mảng a kiểu Vector{Int} dài n phần tử ($0 \le n \le 10000$) và khóa x kiểu số nguyên, không hạn chế miền giá trị. Mảng buộc phải tăng không giảm, tức $a[1] \le a[2] \le \ldots \le a[n]$; điều kiện này là thứ duy nhất khiến tìm nhị phân đúng, nên chương trình kiểm tra nó trước khi làm gì khác. Khóa trùng được phép và chính là trường hợp đề muốn khai thác. Dữ liệu được nhập từ bàn phím hoặc khai báo thẳng trong mã, không đọc tệp CSV/Excel; chỉ số tính từ 1 theo quy ước của Julia.

1.3.2. Kết quả và trường hợp đặc biệt

Đầu ra là bộ ba first, last, count kèm số lần xét mid của từng lần tìm, dùng làm bằng chứng đếm phép toán. Khi x vắng mặt, cả ba bằng 0; khi x có mặt, first ≤ last và count = last − first + 1. Dữ liệu sai (mảng không tăng, n vượt 10000, x không nguyên) bị ném ArgumentError ngay ở khâu kiểm tra; còn mảng rỗng vẫn là đầu vào hợp lệ và cho kết quả 0, 0, 0.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
a (mảng đầu vào)	Vector{Int}, n phần tử	$0 \le n \le 10000$; $a[i] \le a[i+1]$	Khai báo trực tiếp / bàn phím
n (số phần tử)	Số nguyên không âm	$0 \le n \le 10000$	Suy từ độ dài mảng a
x (khóa cần tìm)	Số nguyên	Không giới hạn miền; không yêu cầu x có trong a	Nhập bàn phím / truyền tham số

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
first, last, count	Ba số nguyên 1-based	0,0,0 khi vắng; count = last − first + 1 khi có	Input sai: ArgumentError
Số lần xét mid	Số nguyên không âm	Mỗi lần tìm ≤ $\lceil \log_2(n+1) \rceil$	Mảng rỗng: 0 lần

Mọi dữ liệu đều đi qua bàn phím, khai báo trong mã hoặc tham số truyền vào hàm, không có tệp ngoài nào tham gia. Khái niệm hòa không dùng được ở đây vì kết quả chỉ là hai biên của một đoạn giá trị; còn "vô nghiệm" tương ứng với việc x vắng mặt và được biểu diễn bằng bộ ba 0 thay vì một thông báo lỗi.

1.4. Cách thực hiện và cấu trúc báo cáo

Cách làm đi theo năm bước: đọc mục 2.6.2 của giáo trình, giải tay ví dụ thứ nhất, cài hai hàm biên, kiểm thử kèm đối chiếu quét tuyến tính, rồi chụp lại màn hình làm bằng chứng. Năm chương của báo cáo lần lượt trình bày bối cảnh, cơ sở lý thuyết, phần phân tích và cài đặt, kết quả thử nghiệm, và cuối cùng là kết luận.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	**Sản phẩm/điều kiện kiểm**	**Mục sẽ trình bày**
Hai tìm kiếm nhị phân tìm biên	vi_tri_dau và vi_tri_cuoi chạy đúng mọi ca	Chương 3, mục 3.3
Tần suất và trường hợp vắng	count = last − first + 1; vắng trả 0,0,0	Chương 3, mục 3.5
Đối chứng và đếm phép toán	Khớp quét tuyến tính; đếm số lần xét mid	Chương 4, mục 4.7–4.8

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Hai đối tượng cần bàn là mảng đã sắp a[1..n] và khóa x. Do tính tăng không giảm, mỗi phần tử chỉ rơi vào một trong ba quan hệ với x: nhỏ hơn, bằng, hoặc lớn hơn; ba quan hệ này xếp thành ba vùng liền nhau trên mảng. Vị trí đầu cần tìm chính là biên trái của vùng "bằng x", vị trí cuối là biên phải của chính vùng đó. Giáo trình INT1306 bàn kỹ tìm kiếm nhị phân và các biến thể tìm biên ở mục 2.6.2, trang in 50–52.

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Cấu trúc dữ liệu duy nhất là mảng Vector{Int}, truy cập ngẫu nhiên O(1) nên đọc a[mid] không tốn thêm chi phí. Ngoài mảng, thuật toán chỉ giữ bốn biến nguyên lo, hi, mid, kq cùng hai bộ đếm, không cấp phát mảng phụ nào. Chọn mảng là hợp lý vì thao tác cần thiết chỉ là so sánh a[mid] với x, và dữ liệu đã được sắp sẵn từ trước.

2.2.2. Quy ước và điều kiện hợp lệ

Bốn quy ước cần nhớ: chỉ số bắt đầu từ 1; mảng rỗng Int[] ứng với n = 0; khóa trùng là hợp lệ; và số 0 ở đầu ra luôn mang nghĩa "không thấy", vì mảng không có vị trí 0. Một quy ước nữa mang tính bắt buộc: không được lấy vị trí chèn của x thay cho một lần xuất hiện. Vị trí chèn chỉ là mốc so sánh giữa hai đoạn giá trị, nó không chứng minh a[vị trí] bằng x.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
a[1..n]	Mảng đã tăng không giảm	a[i] ≤ a[i+1]; khóa trùng hợp lệ
lo, hi	Hai đầu vùng còn xét	1 ≤ lo ≤ n+1; 0 ≤ hi ≤ n; vùng rỗng khi lo > hi
mid	Vị trí giữa	mid = lo + (hi − lo) ÷ 2, không tràn số
kq	Vị trí biên tạm	0 hoặc một vị trí có a[kq] = x

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Mỗi vòng lặp so sánh a[mid] với x rồi cắt bỏ một nửa vùng còn xét. Nếu a[mid] < x, mọi vị trí từ mid trở về trước đều nhỏ hơn x nên không thể là biên phải, ta dời lo lên mid + 1. Nếu a[mid] > x, mọi vị trí từ mid trở về sau đều lớn hơn x nên không thể là biên trái, ta hạ hi xuống mid − 1. Trường hợp còn lại a[mid] = x: mid là một ứng viên, ta ghi kq = mid rồi đi tiếp về phía vẫn có thể chứa biên — sang trái với bản tìm đầu, sang phải với bản tìm cuối.

2.3.2. Điều kiện bắt đầu và kết thúc

Trước khi vào lặp, đặt lo = 1, hi = n và kq = 0. Điều kiện chạy là lo ≤ hi; thân vòng tính mid, so sánh, rồi thu hẹp đúng một phía. Khi lo vượt hi thì dừng và trả kq — hoặc là vị trí biên tìm được, hoặc là 0 nếu chưa từng gặp x. Mảng rỗng làm điều kiện sai ngay từ đầu nên kết quả là 0 mà không cần xét mid lần nào.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Chi phí đáng đếm là số lần so sánh a[mid] với x, tương ứng số vòng lặp. Mỗi vòng cắt ít nhất một nửa số ứng viên, nên số vòng không vượt $\lceil \log_2(n+1) \rceil$; hai lần tìm cộng lại vẫn là O(log n), chưa kể bước kiểm tra dữ liệu vào tốn O(n). Bộ nhớ phụ chỉ O(1) vì không có mảng trung gian, còn quét tuyến tính O(n) được giữ riêng cho mục đích đối chứng. Từ đó có thể đoán trước: khi n đi từ 5 lên 5000, số lần xét mid chỉ tăng vài đơn vị.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Hợp đồng của từng hàm, tên biến giữ đúng như trong mã nguồn:

• kiem_tra_dau_vao(a) → true nếu mảng hợp lệ, ngược lại ném ArgumentError.

• vi_tri_dau(a, x) → chỉ số 1-based của lần xuất hiện đầu tiên, 0 nếu vắng.

• vi_tri_cuoi(a, x) → chỉ số 1-based của lần xuất hiện cuối cùng, 0 nếu vắng.

• tan_suat(a, x) → (dau, cuoi, count, xet_dau, xet_cuoi).

• lan_vet_tim_bien(a, x) → in từng bước của cả hai lần tìm và trả kết quả.

• quet_tuyen_tinh(a, x) → bản tham chiếu O(n) để đối chiếu.

• vi_tri_chen(a, x), vi_tri_sau(a, x) → lower/upper bound, chỉ phụ trợ cho phần tùy chọn.

• dem_trong_doan(a, L, U) → yêu cầu tùy chọn (không bắt buộc), đếm phần tử trong đoạn giá trị.

3.1.2. Kiểm tra đầu vào

Bốn điều kiện bị kiểm tra lần lượt: a đúng kiểu Vector{Int}, độ dài nằm trong 0…10000, mảng không giảm, và x là số nguyên. Vi phạm bất kỳ điều nào đều bị ném ArgumentError kèm lý do cụ thể để người dùng biết sửa chỗ nào. Cần phân biệt cho rõ: mảng rỗng không phải dữ liệu sai mà là dữ liệu hợp lệ cho kết quả 0, 0, 0; tương tự, x vắng mặt là một kết quả bình thường chứ không phải lỗi.

3.2. Thiết kế các hàm và dữ liệu

Toàn bộ phần tìm kiếm được viết bằng vòng lặp tay, không gọi searchsorted hay bất kỳ hàm tìm kiếm nào có sẵn trong lõi Julia; những hàm đó chỉ xuất hiện trong kiểm thử với vai trò oracle độc lập. Phần tự cài gồm ba mảnh: vòng lặp thu hẹp vùng, phép tính mid theo dạng lo + (hi − lo) ÷ 2 để tránh tràn, và hai nhánh ghi nhận kq ứng với hai phía cần tìm.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
vi_tri_dau(a, x)	a tăng, x nguyên	Int: vị trí đầu hoặc 0	Lõi tìm biên trái (tự cài)
vi_tri_cuoi(a, x)	a tăng, x nguyên	Int: vị trí cuối hoặc 0	Lõi tìm biên phải (tự cài)
tan_suat(a, x)	a tăng, x nguyên	NamedTuple (dau, cuoi, count, xet_dau, xet_cuoi)	Ghép hai biên, tính tần suất
lan_vet_tim_bien(a, x)	a hợp lệ	In từng bước; trả kết quả	Lần vết T01
quet_tuyen_tinh(a, x)	a hợp lệ	NamedTuple (dau, cuoi, count, so_sanh)	Đối chứng O(n)

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
vi_tri_dau(a, x)	a tăng; x nguyên	0 hoặc vị trí đầu tiên có a[vị trí] = x	Mảng rỗng trả 0
vi_tri_cuoi(a, x)	a tăng; x nguyên	0 hoặc vị trí cuối cùng có a[vị trí] = x	Mảng rỗng trả 0
tan_suat(a, x)	a hợp lệ	count = 0 nếu vắng; ngược lại last − first + 1	Hai biên phải cùng thấy/không thấy x
kiem_tra_dau_vao(a)	a bất kỳ	true khi hợp lệ	Ném ArgumentError khi vi phạm

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

```
      Giả mã vi_tri_dau:
  lo = 1; hi = n; kq = 0
  while lo <= hi
      mid = lo + (hi - lo) // 2
      if a[mid] < x: lo = mid + 1
      elseif a[mid] > x: hi = mid - 1
      else: kq = mid; hi = mid - 1
  return kq
vi_tri_cuoi đối xứng, chỉ đổi nhánh bằng thành lo = mid + 1.
```

3.3.2. Đoạn mã và giải thích

Đoạn mã quyết định nằm ở src/DeTai.jl, gồm vòng while thu hẹp vùng, cách tính mid bằng lo + (hi − lo) ÷ 2 để không tràn số, và nhánh a[mid] = x ghi kq rồi đi tiếp về phía còn khả năng chứa biên. Đường dẫn permalink tới đúng đoạn mã này ở commit C được ghi trong Bảng 4.5.

3.3.3. Xử lý tình huống đặc biệt

Các tình huống đặc biệt được xử lý như sau: n = 0 cho 0, 0, 0 và không xét mid lần nào; n = 1 xét đúng một lần; x nhỏ hơn hoặc lớn hơn mọi phần tử đều cho 0; mảng toàn bằng x cho first = 1 và last = n. Dữ liệu sai bị chặn ngay tại kiem_tra_dau_vao, trước khi vào lõi tìm kiếm. Bốn ca BIEN01–BIEN03 và LOI01 ở Chương 4 kiểm chính những nhánh này.

3.4. Lần vết ví dụ nhỏ

Ca truy vết T01 lấy đúng dữ liệu của Ví dụ 1: a = [1,2,2,2,4] và x = 2, kỳ vọng first = 2, last = 4, count = 3.

Ca truy vết T01 Input đầy đủ [1,2,2,2,4], x = 2 Expected (2, 4, 3)

Bảng 3.3 Truy vết ca

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
Tìm đầu · b1	lo=1, hi=5 → mid=3	a[3]=2 = x	kq=3; hi=2	Ghi nhận 3, tiếp tục sang trái
Tìm đầu · b2	lo=1, hi=2 → mid=1	a[1]=1 < 2	lo=2	Loại nửa trái
Tìm đầu · b3	lo=2, hi=2 → mid=2	a[2]=2 = x	kq=2; hi=1	Ghi nhận 2, tiếp tục sang trái

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
Tìm đầu · kết thúc	lo=2 > hi=1	—	first=2	3 lần xét mid
Tìm cuối · b1	lo=1, hi=5 → mid=3	a[3]=2 = x	kq=3; lo=4	Ghi nhận 3, tiếp tục sang phải
Tìm cuối · b2	lo=4, hi=5 → mid=4	a[4]=2 = x	kq=4; lo=5	Ghi nhận 4
Tìm cuối · b3	lo=5, hi=5 → mid=5	a[5]=4 > 2	hi=4	Loại nửa phải
Tìm cuối · kết thúc	lo=5 > hi=4	—	last=4	3 lần xét mid
Ghép kết quả	first=2, last=4	count = last − first + 1	count=3	Không dùng vị trí chèn

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Bản tìm đầu giữ ba bất biến. Thứ nhất, mọi vị trí nhỏ hơn lo đều có giá trị nhỏ hơn x. Thứ hai, mọi vị trí lớn hơn hi đều có giá trị lớn hơn hoặc bằng x. Thứ ba, kq luôn là ứng viên tốt nhất đã gặp: hoặc bằng 0, hoặc là một vị trí có a[kq] = x mà bên trái nó không còn vị trí nào bằng x. Khi a[mid] = x, mọi vị trí bên phải mid chắc chắn không thể là lần xuất hiện đầu, nên việc hạ hi xuống mid − 1 vẫn giữ nguyên cả ba bất biến. Bản tìm cuối lập luận đối xứng, chỉ đổi chiều cập nhật thành lo = mid + 1.

3.5.2. Kết thúc và output

Vòng lặp chắc chắn dừng vì mỗi vòng hoặc lo tăng hoặc hi giảm ít nhất một đơn vị, trong khi khoảng [lo, hi] luôn bị thu hẹp. Lúc thoát, kq hoặc là biên cần tìm, hoặc bằng 0 khi x không có trong mảng. Tần suất chỉ được tính khi first khác 0, và công thức count = last − first + 1 đúng vì mảng đã tăng nên mọi vị trí nằm giữa hai biên đều mang

giá trị x. Phần kiểm thử ở Chương 4 chỉ bổ sung bằng chứng thực nghiệm, không thay cho lập luận này.

3.6. Phân tích độ phức tạp

Một lần tìm tốn không quá $\lceil \log_2(n+1) \rceil$ lần xét mid, nên hai lần cộng lại không quá $2 \cdot \lceil \log_2(n+1) \rceil$. Tính cả bước kiểm tra dữ liệu vào O(n), chi phí toàn chương trình là O(n) — nghĩa là với đề này bước kiểm tra mới là phần chiếm thời gian, chứ không phải phần tìm. Bộ nhớ phụ vẫn O(1); quét tuyến tính O(n) chỉ tồn tại để đối chứng.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	**Tham số quy mô**	**Phép toán đếm**	**Thời gian**	**Bộ nhớ phụ**
Kiểm tra và chuẩn bị	n	Kiểm tra kiểu, miền, thứ tự	O(n)	O(1)
Tìm vị trí đầu	n	$\leq \lceil \log_2(n+1) \rceil$ lần xét mid	O(log n)	O(1)
Tìm vị trí cuối	n	$\leq \lceil \log_2(n+1) \rceil$ lần xét mid	O(log n)	O(1)
Ghép và tính tần suất	1	Một phép trừ khi first ≠ 0	O(1)	O(1)
Toàn chương trình (có kiểm tra đầu vào)	n	Kiểm tra O(n) + hai lần tìm O(log n)	O(n)	O(1)

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Toàn bộ số liệu trong chương này được lấy trên Windows 11, Julia 1.13.1 chỉ dùng thư viện chuẩn, soạn trong Visual Studio Code 1.142.0 và chạy bằng terminal PowerShell. Mã chính nằm ở src/DeTai.jl, phần demo ở demo.jl, bộ kiểm thử ở test/runtests.jl. Commit C là bản đã chạy để chụp ảnh, commit S là bản chốt bài; cả hai SHA được ghi ở mục 4.9 và trong tệp ThongTinNop.txt.

4.1.2. Phương pháp xác định kỳ vọng

Kỳ vọng của mỗi ca được xác định theo hai đường độc lập: giải tay từng bước, và cho quét tuyến tính kiểm lại trong bộ kiểm thử. Vì kết quả là số nguyên nên chỉ cần so sánh bằng, không phải bàn tới sai số. Mỗi ca đều ghi đủ bốn thứ: dữ liệu vào đủ để tái hiện, kỳ vọng, kết quả chạy thật và kết luận PASS/FAIL.

4.2. Bảng kết quả 13 ca

Mười ba ca có mã được liệt kê ở bảng dưới, chia thành bốn nhóm: năm ca lấy từ phiếu (hai ví dụ và ba ca biên), hai ca tự thiết kế, một ca dữ liệu sai và năm ca bổ sung cho các trường hợp còn thiếu. Ngoài ra bộ kiểm thử còn sinh mảng ngẫu nhiên với seed cố định để kiểm bất biến trên nhiều hình dạng dữ liệu.

Phần lớn ca vẫn xoay quanh mảng số nguyên đã sắp. Những ca còn lại nhắm vào hàm phụ — validator, quét tuyến tính, đếm trong đoạn — để chạm tới các nhánh mà dữ liệu bình thường hiếm khi tạo ra.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	[1,2,2,2,4], x = 2 → 2, 4, 3	VD01 / A01	01_vi_du_1.png
Ca phiếu 2	Cùng mảng, x = 3 → 0, 0, 0	VD02 / A02	02_vi_du_2.png

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 3	[5,5,5], x = 5 → 1, 3, 3	BIEN03 / A07	07_bien_3.png
Ca phiếu 4	[1,2], x = 1 → 1, 1, 1	BIEN01 / A05	05_bien_1.png
Ca phiếu 5	Mảng rỗng → 0, 0, 0	BIEN02 / A06	06_bien_2.png

Năm ca của phiếu giữ nguyên dữ liệu vào và kỳ vọng như phiếu ghi; ca nào trùng mục tiêu thì dùng chung một mã ảnh nhưng vẫn được kiểm riêng trong bộ test. Hai ca đầu cố ý dùng cùng một mảng để người đọc thấy ngay khác biệt giữa trường hợp x có mặt và x vắng mặt.

Bảng 4.2 Thiết kế 13 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01	Ví dụ 1 phiếu / tan_suat	[1,2,2,2,4], x=2	first=2, last=4, count=3; xét mid=6
VD02	Ví dụ 2 phiếu / tan_suat	[1,2,2,2,4], x=3	0,0,0; xét mid=6; không nhận vị trí chèn
TK01	Tự thiết kế 1 / vi_tri_dau	[1,1,1,5,9], x=1	biên trái ở 1; count=3
TK02	Tự thiết kế 2 / vi_tri_cuoi	[0,3,7,7], x=7	biên phải ở 4; count=2
BIEN01	Ca phiếu 4 / tan_suat	[1,2], x=1	1,1,1; count=1
BIEN02	Ca phiếu 5 / tan_suat	Int[], x=5	0,0,0; 0 lần xét mid
BIEN03	Ca phiếu 3 / tan_suat	[5,5,5], x=5	1,3,3; toàn mảng bằng x
LOI01	Input sai / kiem_tra_dau_vao	[3,1,2], x=1	ném ArgumentError (mảng không tăng)
KT09	Một phần tử / tan_suat	[5], x=5	1,1,1; xét mid=2
KT10	Nhỏ hơn mọi phần tử / tan_suat	[2,4,6], x=1	0,0,0; lo vượt hi
KT11	Lớn hơn mọi phần tử / tan_suat	[2,4,6], x=9	0,0,0; lo vượt hi (lo = n+1)
KT12	Khóa trùng giữa / tan_suat	[1,3,3,3,3,5,7,7], x=3	first=2, last=5, count=4
KT13	Số âm / tan_suat	[-5,-5,-2,0], x=-5	first=1, last=2, count=2

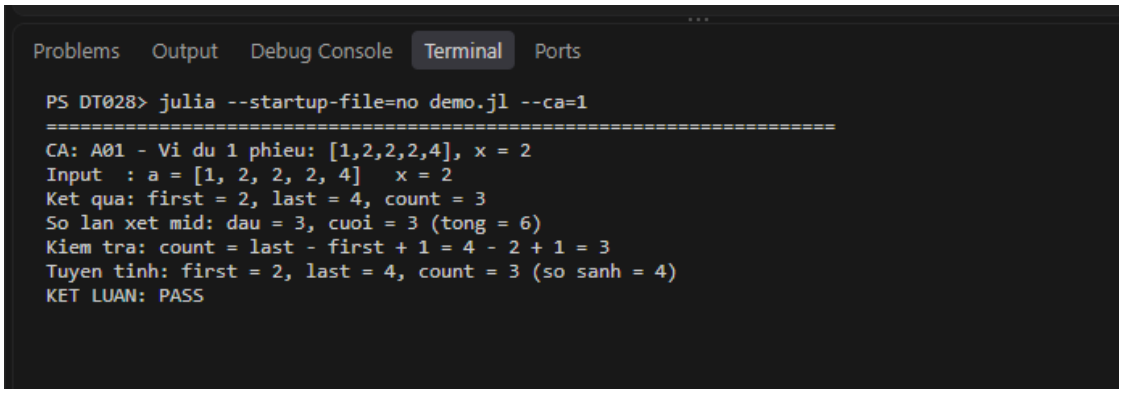
Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	(2, 4, 3)	PASS	A01 / 01_vi_du_1.png
VD02	(0, 0, 0)	PASS	A02 / 02_vi_du_2.png
TK01	(1, 3, 3)	PASS	A03 / 03_tu_thiet_ke_1.png
TK02	(3, 4, 2)	PASS	A04 / 04_tu_thiet_ke_2.png
BIEN01	(1, 1, 1)	PASS	A05 / 05_bien_1.png
BIEN02	(0, 0, 0)	PASS	A06 / 06_bien_2.png
BIEN03	(1, 3, 3)	PASS	A07 / 07_bien_3.png
LOI01	ArgumentError	PASS	A08 / 08_input_khong_hop_le.png
KT09	(1, 1, 1)	PASS	09_tong_ket_kiem_thu.png
KT10	(0, 0, 0)	PASS	09_tong_ket_kiem_thu.png
KT11	(0, 0, 0)	PASS	09_tong_ket_kiem_thu.png
KT12	(2, 5, 4)	PASS	09_tong_ket_kiem_thu.png
KT13	(1, 2, 2)	PASS	09_tong_ket_kiem_thu.png

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

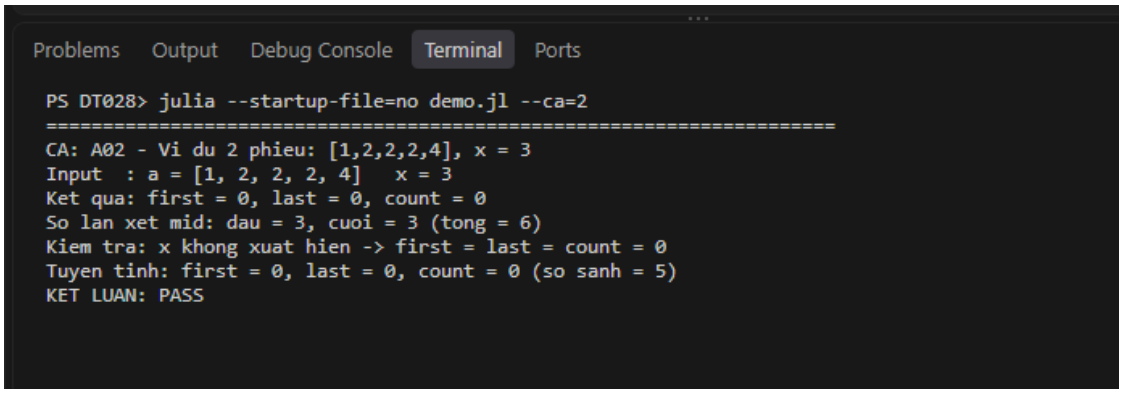
Dữ liệu vào là [1,2,2,2,4] với x = 2; kỳ vọng first = 2, last = 4, count = 3, tính bằng cách giải tay. Lệnh chạy: julia --startup-file=no demo.jl --ca=1. Kết quả thật là (2, 4, 3) sau 6 lần xét mid, và quét tuyến tính cho cùng bộ số.



Hình 4.1 — VD01: kết quả in ra từ demo.jl --ca=1, chụp trong VS Code.

4.3.2. Ví dụ 2 — VD02

Vẫn mảng [1,2,2,2,4] nhưng lấy x = 3; vì 3 không có trong mảng nên kỳ vọng là (0,0,0). Lệnh chạy: julia --startup-file=no demo.jl --ca=2. Kết quả thật đúng là (0,0,0) sau 6 lần xét mid. Giá trị của ca này nằm ở chỗ nó tách bạch vị trí chèn (ngay sau các phần tử 2) với một lần xuất hiện thật.

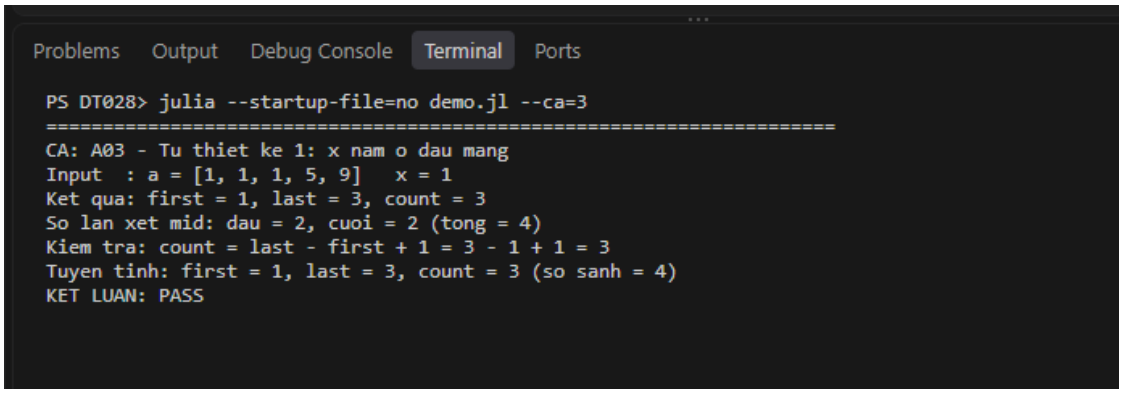


Hình 4.2 — VD02: x vắng mặt nên kết quả là 0, 0, 0.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

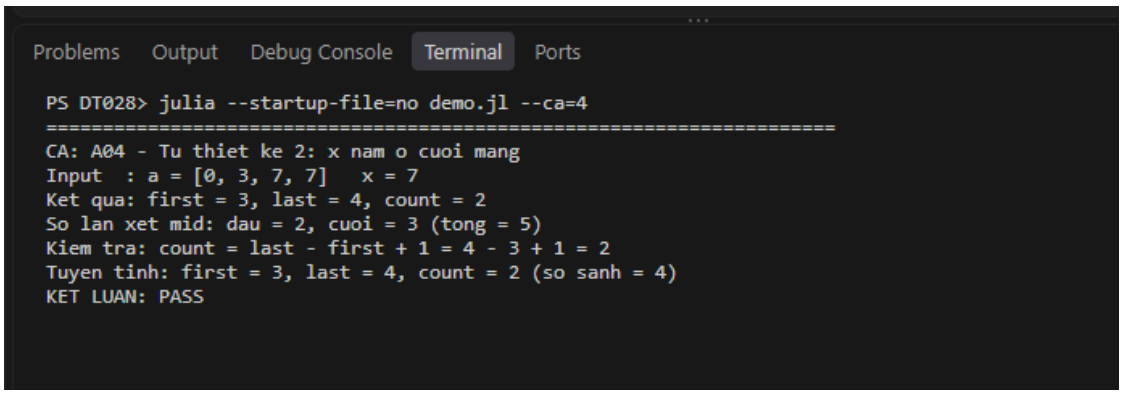
Ca này nhắm vào biên trái khi x đứng ngay đầu mảng. Dữ liệu [1,1,1,5,9] với x = 1 cho kỳ vọng first = 1, last = 3, count = 3. Chạy thật khớp kỳ vọng, cả hai lần tìm đều tốn 4 lần xét mid.



Hình 4.3 — TK01: x nằm ở đầu mảng.

4.4.2. Ca TK02

Ca này nhắm vào biên phải khi x nằm ở cuối mảng. Dữ liệu [0,3,7,7] với x = 7 cho kỳ vọng first = 3, last = 4, count = 2. Kết quả thật khớp; đáng chú ý là bản tìm cuối buộc phải xét thêm mid = 4 mới chốt được biên.

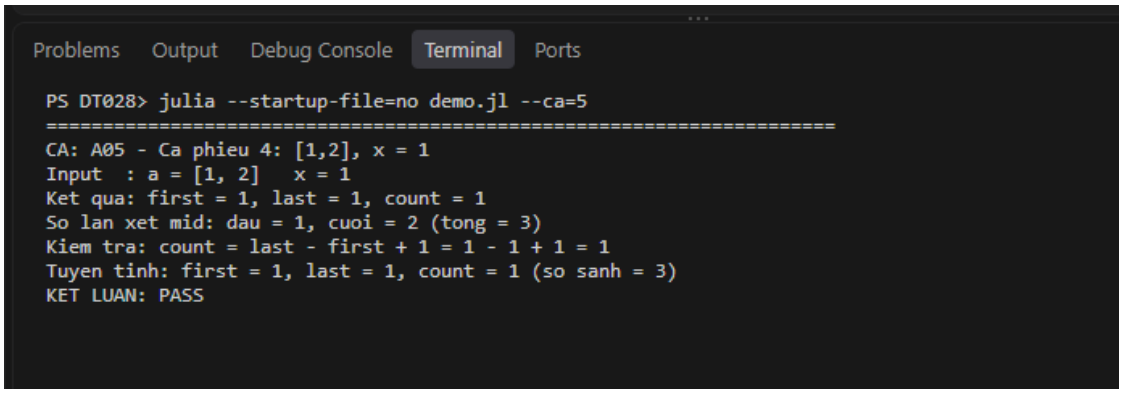


Hình 4.4 — TK02: x nằm ở cuối mảng.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

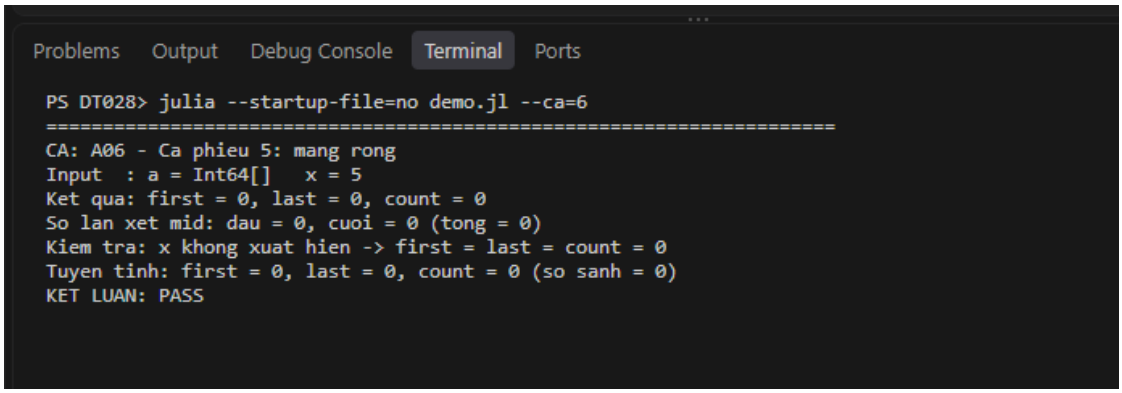
Ca biên lấy từ phiếu: mảng [1,2] với x = 1. Kỳ vọng (1,1,1) vì x đứng ngay đầu mảng và không có khóa trùng nào. Kết quả thật khớp; bản tìm đầu chỉ cần 1 lần xét mid, bản tìm cuối cần 2 lần.



Hình 4.5 — BIEN01: mảng chỉ có hai phần tử.

4.5.2. Ca BIEN02

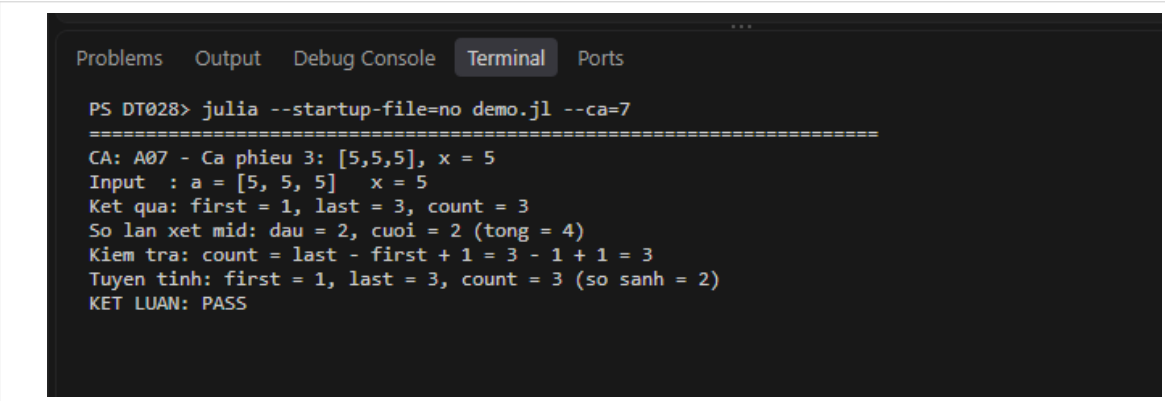
Ca biên thứ hai là mảng rỗng Int[] với x = 5, tức ca 5 của phiếu. Kỳ vọng (0,0,0) và không xét mid lần nào. Kết quả thật khớp, xác nhận rằng chương trình không hề truy cập a[mid] khi n = 0.



Hình 4.6 — BIEN02: mảng rỗng.

4.5.3. Ca BIEN03

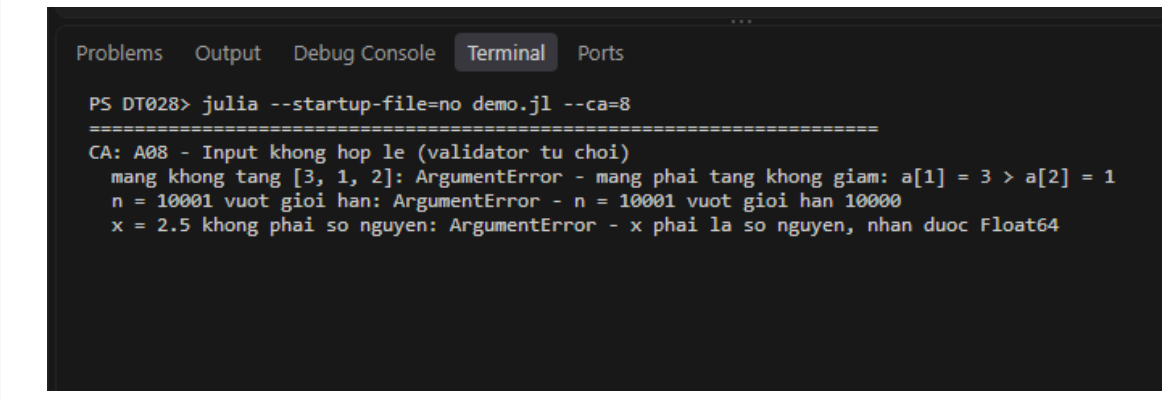
Ca biên thứ ba cho toàn mảng bằng x: [5,5,5] với x = 5, tức ca 3 của phiếu. Kỳ vọng (1,3,3) vì hai biên trùng với hai đầu mảng và tần suất bằng n. Kết quả thật khớp; đây là trường hợp vùng bằng x phủ kín cả mảng.



Hình 4.7 — BIEN03: mọi phần tử đều bằng x.

4.6. Kiểm tra input không hợp lệ — LOI01

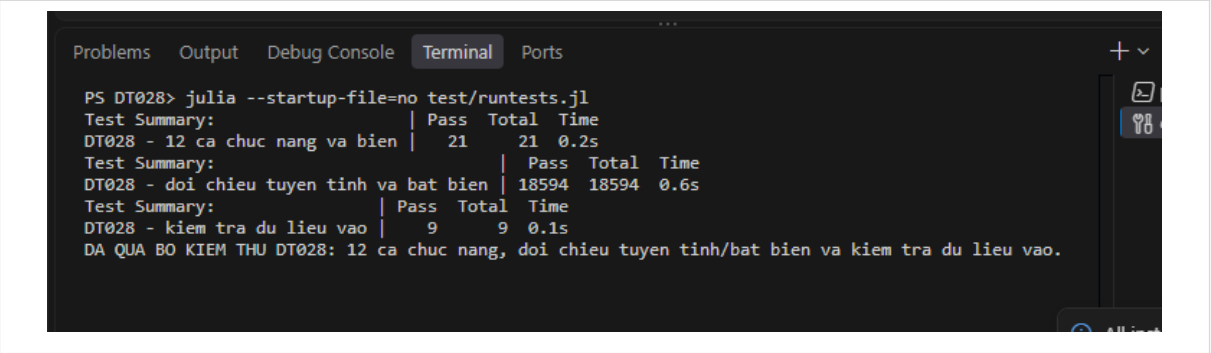
Ba kiểu vi phạm được thử: mảng [3,1,2] không tăng, n = 10001 vượt mức 10000, và x = 2.5 không phải số nguyên. Kỳ vọng là kiem_tra_dau_vao hoặc bước kiểm tra x ném ArgumentError kèm lý do, và lõi tìm kiếm không được gọi tới. Kết quả thật: cả ba trường hợp đều bị chặn đúng như vậy.



Hình 4.8 — LOI01: ba kiểu dữ liệu sai đều bị từ chối.

4.7. Kết quả toàn bộ kiểm thử

Lần chạy lại toàn bộ kiểm thử cho 18.624 phép kiểm, tất cả đều đạt, chia làm ba nhóm: 21 phép ở nhóm chức năng (12 ca chức năng cộng ca lỗi LOI01), 18.594 phép ở nhóm đối chiếu bất biến trên mảng ngẫu nhiên, và 9 phép ở nhóm kiểm tra dữ liệu vào. Không còn ca nào FAIL.



Hình 4.9 — Tổng kết: 18.624 trên 18.624 phép kiểm đạt.

4.8. Khảo sát bốn quy mô hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Bốn quy mô được khảo sát: n = 5, 50, 500 và 5000, đều dùng mảng tăng có nhiều khóa trùng và chọn x ở giữa mảng để chắc chắn x có mặt. Đại lượng đem so là số lần xét mid của từng biên, tổng của hai lần tìm, và số phép so sánh của quét tuyến tính. Thời gian chỉ đo để tham khảo bằng time_ns, lấy giá trị tốt nhất trong 5 lần chạy.

4.8.2. Kết quả và giới hạn nhận xét

Trong khi n tăng 1000 lần, tổng số lần xét mid chỉ đi từ 5 lên 12, 18 rồi 25, đúng dáng tăng theo $\log_2(n)$. Quét tuyến tính thì tăng gần như tuyến tính: 6, 50, 480 và 4.781 phép so sánh. Các con số này nằm dưới trần $2 \cdot \lceil \log_2(n+1) \rceil$, qua đó xác nhận O(log n) cho hai lần tìm.

Bảng 4.4 Khảo sát Q1–Q4

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	n = 5, mảng tăng có trùng	quy mô n	xét mid: 3 + 2 = 5; tuyến tính 6	O(1) phụ; khớp trần 6
Q2	n = 50, mảng tăng có trùng	quy mô n	xét mid: 6 + 6 = 12; tuyến tính 50	O(1) phụ; khớp trần 12
Q3	n = 500, mảng tăng có trùng	quy mô n	xét mid: 9 + 9 = 18; tuyến tính 480	O(1) phụ; khớp trần 18
Q4	n = 5000, mảng tăng có trùng	quy mô n	xét mid: 13 + 12 = 25; tuyến tính 4.781	O(1) phụ; dưới trần 26

Mảng khảo sát được sinh bằng Random.seed!(28062026) nên ai chạy lại cũng ra cùng dữ liệu. Số đo thời gian chỉ mang tính minh hoạ và không được dùng để chứng minh bậc phức tạp; bằng chứng chính vẫn là phép đếm.

Ở n = 5000, quét tuyến tính tốn 4.781 phép so với 25 lần xét mid, tức gấp gần 191 lần — đây là lợi ích rõ nhất của tìm nhị phân trên mảng lớn. Ngược lại ở n = 5, quét tuyến tính có thể nhanh hơn nhờ hằng số nhỏ, nên bài chỉ so bậc tăng trưởng chứ không so thời gian tuyệt đối.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Commit C, URL và cách kiểm lại

Kho mã công khai nằm ở https://github.com/Mildsivan/ctdl-baocao. Bản đã dùng để chạy và chụp ảnh là commit C, SHA 4277eb8768fbb63ed709df060ecd56e0073346c1; cả URL kho lẫn permalink tới tệp đều đã mở thử trên trình duyệt. Bảng 4.5 gom lại URL, SHA C và permalink để người chấm kiểm nhanh, còn SHA S của bản chốt bài nằm trong ThongTinNop.txt. Hai ảnh A10 và A11 được đặt ở Phụ lục E.

Mã nguồn: https://github.com/Mildsivan/ctdl-baocao

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị kiểm lại
URL repo	https://github.com/Mildsivan/ctdl-baocao
Full SHA chạy ảnh C	4277eb8768fbb63ed709df060ecd56e0073346c1
SHA S ghi TXT nộp rời	Ghi trong ThongTinNop.txt (nộp rời, sau commit S)
Permalink hàm chính	https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/src/DeTai.jl
Mã/kiểm thử giữa C và S	Không đổi; đã chạy lại từ clone C, 13/13 ca đạt.
Lệnh chạy lại	julia --startup-file=no demo.jl ; julia --startup-file=no test/runtests.jl

4.9.2. Chạy lại trên bản sao độc lập

Bản sao được clone từ chính kho trên vào một thư mục trống, sau đó chạy lại toàn bộ quy trình để chắc rằng gói nộp không thiếu tệp nào. Ảnh A10 ghi lại lần chạy đó, ảnh A11 là trang commit C trên GitHub; cả hai nằm ở Phụ lục E. Giữa C và S, phần mã, kiểm thử và dữ liệu vào không thay đổi, nên kết quả chạy lại trùng với kết quả đã báo.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

So với bảy mục tiêu ở Chương 1, cả bảy hàm vi_tri_dau, vi_tri_cuoi, tan_suat, lan_vet_tim_bien, kiem_tra_dau_vao, quet_tuyen_tinh và dem_trong_doan đều đã cài và chạy đúng. Mười ba ca có mã đều PASS (Bảng 4.2, 4.3); bốn quy mô khớp O(log n) (Bảng 4.4); kho mã và ảnh minh chứng đã đủ (mục 4.9, Phụ lục E).

5.2. Giới hạn và bài học

Giới hạn nằm ở giả thiết: thuật toán chỉ đúng khi mảng đã sắp, còn nếu phải sắp trước thì chi phí O(n log n) lớn hơn hẳn phần tìm. Một lỗi dễ mắc là lấy vị trí chèn làm first khi x vắng, khiến count sai; ca VD02 được đặt ra để bắt lỗi này, và bài học là phải xác nhận x có mặt rồi mới ghép hai biên.

5.3. Hướng phát triển phù hợp

Ba hướng đi tiếp: hoàn thiện phần tùy chọn đếm phần tử trong đoạn [L, U] đã có trong dem_trong_doan; mở rộng sang mảng giảm hoặc mảng xoay vòng, nơi chỉ cần đổi chiều cập nhật biến; và thử tìm kiếm nội suy khi dữ liệu phân bố đều. Hai hướng sau mới là đề xuất, chưa có mã.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Hai tìm kiếm biên và tần suất	Hoàn thành	Chương 3, Hình 4.1 –4.7	Yêu cầu mảng đã sắp
Đối chứng, đếm phép toán, 4 quy mô	Hoàn thành	Bảng 4.4, log trace	—
Bộ test ≥12 ca, ảnh, mô phỏng HTML	Hoàn thành	Bảng 4.2–4.3; A01–A09	A10–A11 tại Phụ lục E, Hình 4.10–4.11

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; Chương 2, Mục 2.6.2; trang in 50–52; trang PDF 62–64.

[2] Gói mã mẫu BFS của giảng viên (Ma_BFS_mau.zip): chỉ dùng để xem cách chia module, cách viết demo và cách chụp ảnh minh chứng; phần thuật toán và kiểm thử của DT028 do em tự viết.

Cả hai nguồn trên đều được dẫn lại trong nội dung; riêng nguồn mã mẫu chỉ ghi phần học được về cách tổ chức, không dùng lại mã.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Cây tệp kho mã:

src/DeTai.jl — các hàm tìm biên, tần suất, validator, đối chứng

demo.jl — 8 ca demo + so sánh + nhập bàn phím

test/runtests.jl — 12 ca + 18.594 bất biến + kiểm tra input

scripts/test_catalog.jl, scripts/trace_and_scales.jl

docs/MoPhong_TimBien_DT028.html — trang mô phỏng tương tác

Run_all.ps1 — chạy toàn bộ một lệnh

docs/anh_ket_qua, docs/logs — ảnh và log bằng chứng

Yêu cầu: Julia 1.13.1. Lệnh chạy: julia --startup-file=no demo.jl và test/runtests.jl.

PHỤ LỤC B. Danh mục bằng chứng

Danh mục dưới gom đủ mười một ảnh A01–A11, kèm mã ca và tên tệp để đối chiếu với gói ZIP. Chín ảnh đầu nằm trong thân báo cáo, hai ảnh A10 và A11 đặt ở Phụ lục E (Hình 4.10 và 4.11).

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01	01_vi_du_1.png
A02	VD02	02_vi_du_2.png
A03	TK01	03_tu_thiet_ke_1.png
A04	TK02	04_tu_thiet_ke_2.png
A05	BIEN01	05_bien_1.png
A06	BIEN02	06_bien_2.png
A07	BIEN03	07_bien_3.png
A08	LOI01	08_input_khong_hop_le.png
A09	TEST_ALL	09_tong_ket_kiem_thu.png
A10	CLONE	10_clone_sach_chay_lai.png — Hình 4.10, Phụ lục E
A11	GITHUB	11_github_repo_commit.png — Hình 4.11, Phụ lục E

PHỤ LỤC C. Tự kiểm trước khi nộp

Phần này đối chiếu tám mục trong checklist của hướng dẫn: thông tin cá nhân và mã đề, số ca kiểm thử, số ảnh, số quy mô khảo sát, URL/SHA/permalink, bản clone chạy lại, và sự khớp giữa DOCX với PDF. Đây chỉ là tự xác nhận sản phẩm đã chuẩn bị đủ, không phải tự chấm điểm.

□ MSSV và đề tài đúng bảng phân công (N24DCDT034 / DT028).

□ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.

□ 13 ca độc lập, đủ ví dụ và ca phiếu; ảnh A01–A11 đều chụp từ lần chạy thật trong VS Code.

□ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và bốn quy mô khảo sát.

□ Đã có A10/A11, URL, SHA C và permalink; SHA S ghi trong ThongTinNop.txt.

□ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tên tệp đặt theo mẫu N24DCDT034_DT028_PhanDucHuy: viết không dấu, không khoảng trắng, đúng bốn tệp.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT034_DT028_PhanDucHuy_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	N24DCDT034_DT028_PhanDucHuy_BaoCao.pdf	Xuất từ chính bản Word nộp.
Mã nguồn	N24DCDT034_DT028_PhanDucHuy_MaNguon.zip	Source, demo, test, README, ảnh/log, mô phỏng HTML.
Thông tin	N24DCDT034_DT028_PhanDucHuy_ThongTinNop.txt	URL GitHub, SHA C/S, permalink, lệnh.

Không được lấy nguyên gói mẫu của giảng viên rồi nộp như ZIP cá nhân. ZIP của mình phải giải nén ra thư mục mới và chạy được theo README; đồng thời không chứa .git, bộ cài, môi trường ảo hay cache.

Hai mốc cần phân biệt: C là lúc mã, test và dữ liệu đã đứng yên để chụp ảnh; S là lúc chốt toàn bộ bài nộp. Muốn kiểm thì clone S sang một thư mục trống rồi chạy lại; trường hợp mã bị sửa sau C thì test và ảnh liên quan cũng phải làm lại. SHA của S nằm trong tệp TXT nộp kèm.

Các thông tin để chép vào TXT

URL GitHub thật: https://github.com/Mildsivan/ctdl-baocao

SHA C đầy đủ: 4277eb8768fbb63ed709df060ecd56e0073346c1 SHA S: ghi trong ThongTinNop.txt (nộp rời)

Permalink hàm chính tại C: https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/src/DeTai.jl ; test: https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/test/runtests.jl

Môi trường và phiên bản: Julia 1.13.1, VS Code 1.142.0, Windows 11

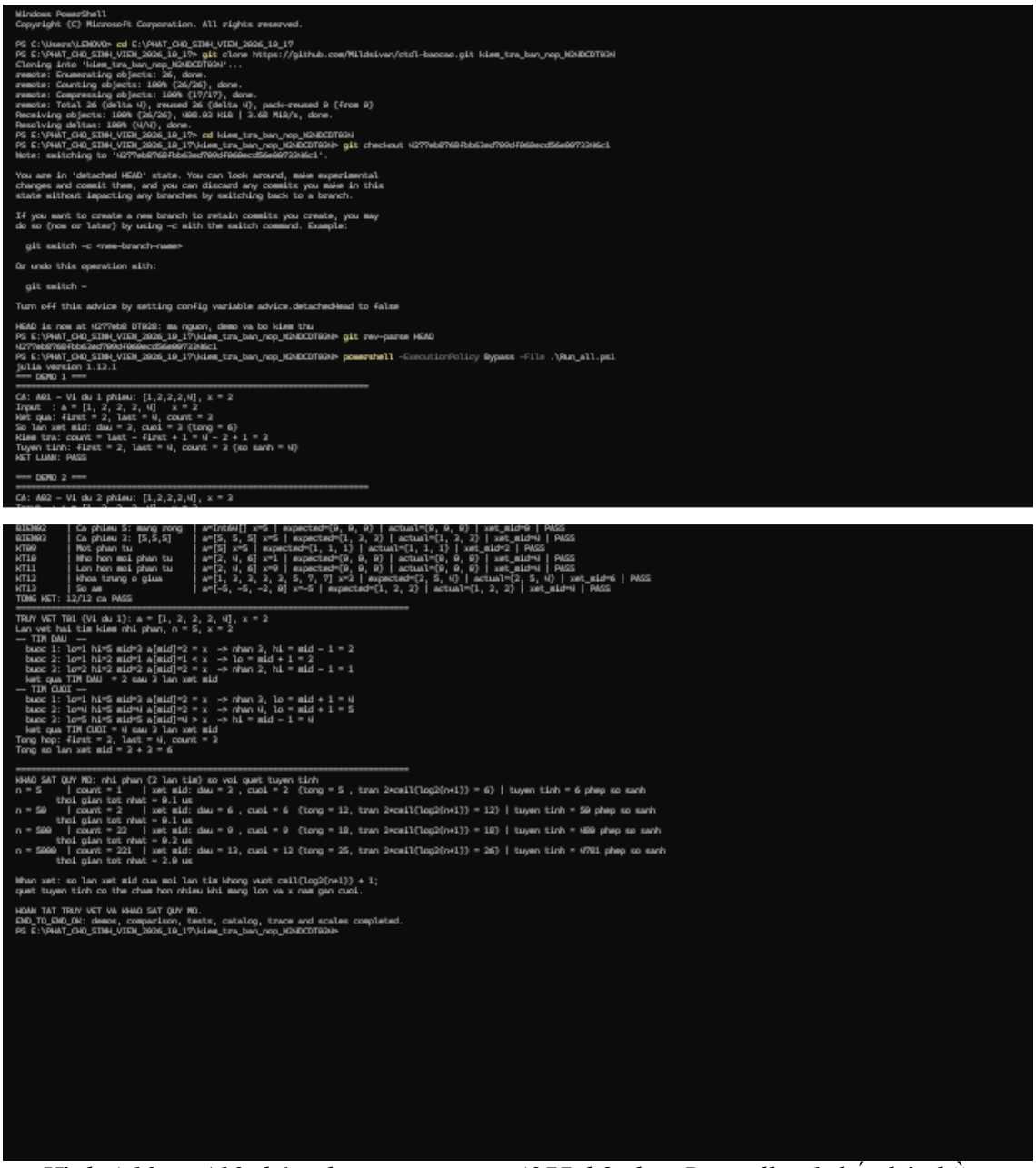
Chạy thử: julia --startup-file=no demo.jl --ca=1 rồi --ca=2; sau đó julia --startup-file=no test/runtests.jl. Cả ba lệnh đều dùng cờ --startup-file=no để không nạp cấu hình cá nhân.

Kết quả clone S: ghi trong ThongTinNop.txt (sau khi chốt S) Số ca và phép kiểm: 13 ca / 18.624 phép, 13 PASS

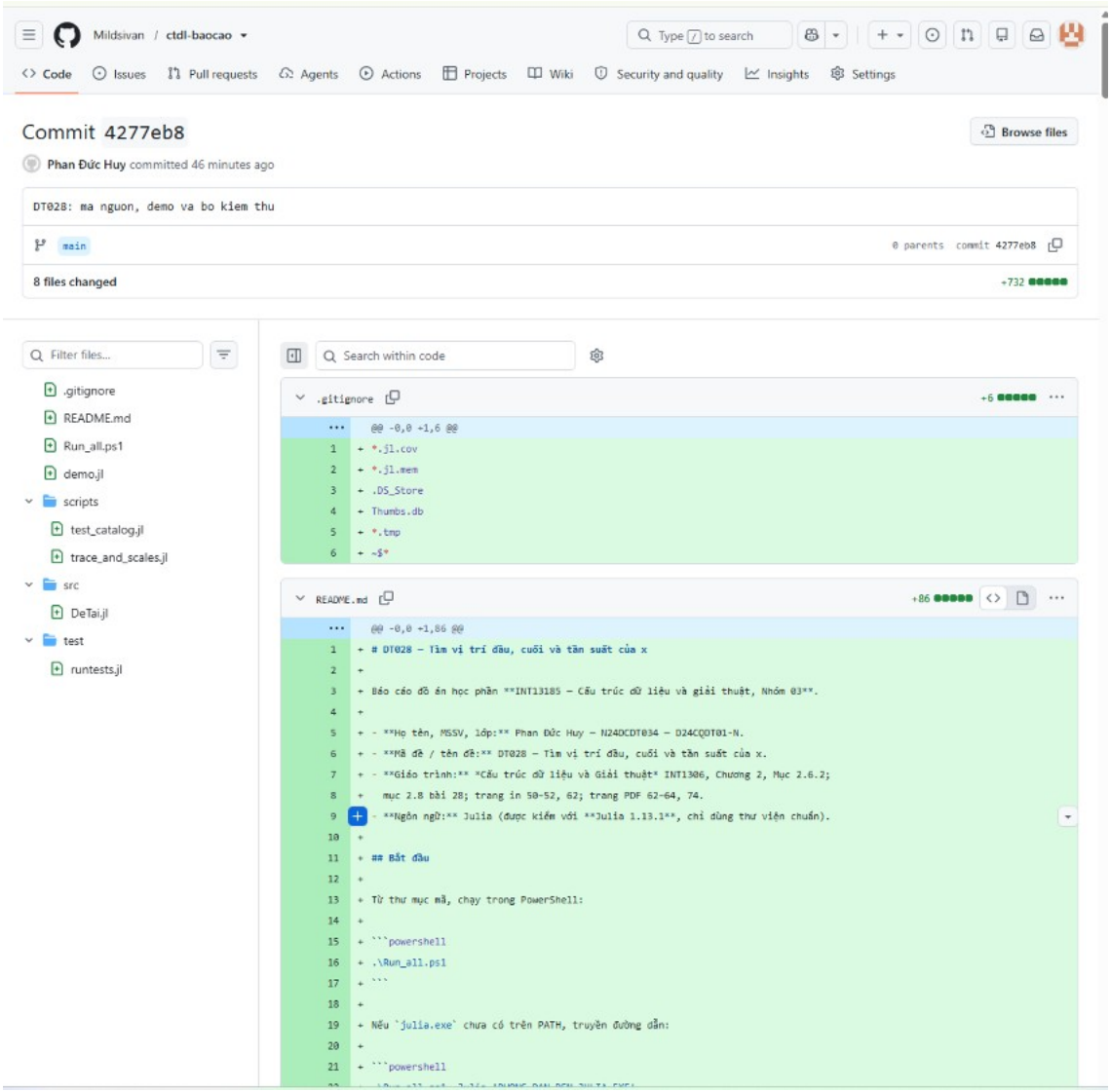
Trên Classroom: mở đúng bài tập, đính kèm đủ bốn tệp, kiểm lại rồi bấm Nộp. Ảnh đã nằm trong báo cáo và ZIP nên không cần gửi lẻ.

PHỤ LỤC E — HAI ẢNH A10 VÀ A11

Hai ảnh dưới được chụp từ lần chạy thật, không phải ảnh dựng: A10 là màn hình sau khi clone commit C và chạy lại toàn bộ quy trình, A11 là trang kho GitHub tại đúng commit đó.



Hình 4.10 — A10: bản clone tại commit 4277eb8 chạy Run_all.ps1, kết thúc bằng END_TO_END_OK.



Hình 4.11 — A11: kho Mildsivan/ctdl-baocao mở đúng tại commit 4277eb8.